BombLab 实验报告

姓名：俞凡　学号：25301980234

完整实现见 defuser.c。我实际运行了单关测试、额外 seed 验证和总测试，截图放在对应关卡及结果汇总中。

一、实验内容与环境

本实验在 VMware 的 Ubuntu x86-64 虚拟机中进行，通过 Windows 的 VS Code Remote SSH 编辑和调试。实验目录为 `/home/yufan/ICS/BombLab`，使用 GCC、Make、GDB 及课程提供的公开测试脚本。当前代码只在 `defuser.c` 中实现求解逻辑，个人炸弹、驱动、接口和测试程序保持原样。

实验的输入不是一组固定口令。驱动根据 seed 和关卡生成 64 位的 `challenge`，求解函数接收它并计算口令，再由使用相同参数的 `bomb` 验证。不同 seed 会生成不同题目，因此需要恢复通用计算关系，不能只保存某次运行得到的答案。

分析时先看输入检查，确定整数个数，再沿调用和比较寻找每个输出的来源。下面只选关键汇编，不逐行翻译整份程序。AT&T 语法中源操作数在前、目的操作数在后；参数和返回值要结合当前调用观察，不能假定一个寄存器在整个函数里始终保存同一个变量。

以下公式中，c 表示 challenge，h 表示 context，并记

$$[x]_{32} = x \bmod 2^{32}, \qquad 0 \le [x]_{32} < 2^{32}.$$

符号 $\oplus$ 表示逐位异或，& 表示逐位与；$x \gg n$ 表示无符号右移，即 $\lfloor x/2^n \rfloor$。除特别标明外，中间量都对应代码中的 `uint32_t`，加法、乘法和左移的结果均保留低 32 位。这样既与汇编写入 `eax` 等 32 位寄存器的行为一致，也避免依赖 C 的有符号溢出。

二、逐关分析与实现

第 1 关：跟踪 helper 的参数和返回值

`phase_01` 的输入检查要求一个整数。它将 challenge 交给 `p1_compute`，再把返回的 `eax` 与这个整数比较。本关的重点是跟清 helper 的实际参数，而不是根据函数名字猜调用顺序。

以下指令分别来自三个 helper 和调用点：

```
402024: lea (%rdi,%rsi,4),%eax
402027: xor $0x5a0,%eax
402034: lea 0x46fa(%rsi),%eax
40203a: xor %edi,%eax
402044: xor $0xfb73,%esi
40204a: lea (%rsi,%rdi,8),%eax
40206d: call 402040 <p1_step2>
40207a: call 402020 <p1_step0>
402083: call 402030 <p1_step1>
```

三个 helper 对应的计算关系分别为

$$
\begin{aligned}
f_0(x,y) &= [x + 4y]_{32} \oplus \texttt{0x5a0}, \\
f_1(x,y) &= [y + \texttt{0x46fa}]_{32} \oplus x, \\
f_2(x,y) &= [(y \oplus \texttt{0xfb73}) + 8x]_{32}\,.
\end{aligned}
$$

`p1_compute` 先以 high、low 为参数调用 step2，再以 low、mid 调用 step0，最后把 a、b 交给 step1。令 c_ℓ、c_m、c_h 分别对应代码中的 low、mid、high，则最终口令为

$$
\begin{aligned}
c_\ell &= [c]_{32}\,, \qquad c_m = [c \gg 16]_{32}\,, \qquad c_h = [c \gg 32]_{32}\,, \\
a &= [c_\ell + 4c_m]_{32} \oplus \texttt{0x5a0}, \\
b &= [(c_\ell \oplus \texttt{0xfb73}) + 8c_h]_{32}\,, \\
v &= a \oplus [b + \texttt{0x46fa}]_{32}\,.
\end{aligned}
$$

这里的 `mid` 与 low、high 有重叠，并不是把 64 位数分成三个互不相交的部分。`lea` 在这几处只进行整数表达式计算，不读取所算地址的内容。实现直接保留 low、mid、high、a、b，最后用 `write_answer` 输出一个数，便于逐项与汇编核对。

使用 seed 42 运行第 1 关，终端输出 `Phase 1 defused.`，随后执行单关公开测试，10 个 seed 全部通过。这里的 seed 42 属于公开列表。



图 1：运行第 1 关，seed 为 42，程序输出 `Phase 1 defused.`

```
yufan@yufan-VMware-Virtual-Platform:~/ICS/BombLab$ python3 public_test
.py --phase 1
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 2：第 1 关公开测试通过 10/10。

```
yufan@yufan-VMware-Virtual-Platform:~/ICS/BombLab$ ./defuser --seed 8
--phase 1 | ./bomb --seed 8 --phase 1
BOMBLAB 2026                                    这里还没有明天
                        *
                         \
                      .-"""-.
                    .'       '.
                   /    LOOP    \
                  |               |
                   \             /
                    '-----------'
[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
单关调试：只记录本次通过的关卡，不播放完整结局。
第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。
Phase 1 challenge: 0x9ca8a164477d7801
Input (unsigned decimal integers):
Phase 1 defused.
[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

图 3：第 1 关使用非公开 seed 8 运行，输出 `Phase 1 defused.`

第 2 关：从栈上数组恢复递推关系

输入检查和 `sscanf` 的返回值比较都要求六个整数。六个槽从 `rbp-0x40` 开始，相邻地址差 4 字节，与 32 位整数数组一致。第一项先从 challenge 取位，后面五项在循环中验证。

```
402160: shr $0x5,%rax
402164: and $0x1f,%eax
402167: add $0x10,%eax
40217b: shr %cl,%rax
40217e: and $0x1f,%eax
402181: xor %edx,%eax
402183: lea (%rax,%rsi,2),%eax
402186: mov -0x40(%rbp,%rdx,4),%esi
40218a: cmp %eax,%esi
402192: add $0x2,%ecx
```

循环开始时 $i = 1$、移位量为 10；每轮 i 增加 1，移位量增加 2，所以移位量为 $8 + 2i$。`lea` 执行时，`esi` 还是上一项，随后程序才从数组读取当前项并比较。六个整数满足

$$v_0 = ((c \gg 5)\ \&\ 31) + 16,$$
$$v_i = \left[2v_{i-1} + \left(((c \gg (8 + 2i))\ \&\ 31) \oplus i\right)\right]_{32}, \quad i = 1, \ldots, 5.$$

实现用六元素数组和一个循环，逐项生成，不把六项写成重复代码。判断顺序很重要：如果把读取当前项误当成发生在 `lea` 之前，就会把递推关系读反。循环结束条件也表明总共生成六项，而不是再额外计算第七项。

第 2 关公开测试通过 10/10。又使用公开列表之外的 seed 8 单独运行第 2 关，输出 `Phase 2 defused.`，截图中的 seed 是 8。

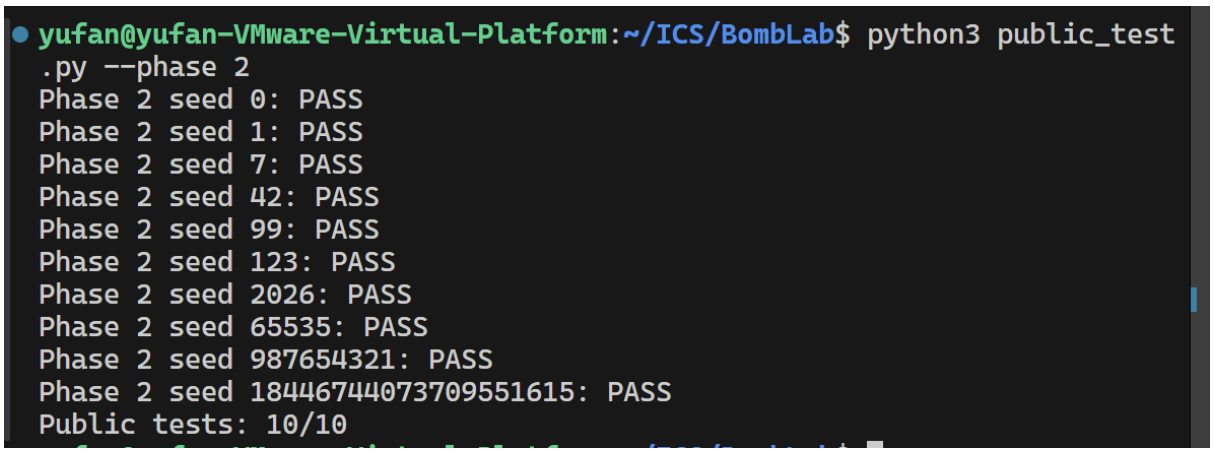


图 4：第 2 关公开测试通过 10/10。



图 5：使用非公开 seed 8 验证第 2 关，程序输出 `Phase 2 defused.`

第 3 关：按跳转表恢复八个分支

本关需要两个整数。第一个整数由 challenge 的低三位决定；第二个整数则由同一个索引选择计算分支。入口先构造传给分派函数的 x，分派函数再从地址表跳到实际代码。

```
4022f1: and $0x7,%edi
4022f4: cmp %edi,-0x18(%rbp)
4022ff: shr $0x3,%rax
402303: shr $0x20,%rsi
402307: xor %eax,%esi
402309: call 402240 <p3_dispatch>
402248: cmp $0x7,%eax
40224b: ja 402292 <p3_dispatch+0x52>
40224d: notrack jmp *0x405820(,%rax,8)
```

由入口指令可得分支索引和参与计算的数：

$$i = c \,\&\, 7, \qquad x = [c \gg 3]_{32} \oplus [c \gg 32]_{32}\,.$$

表项占 8 字节，存的是代码地址；应先读表内目标，再追踪该地址处的跳转。实际索引顺序与 `p3_case_N` 的编号不同，直接照函数名顺序填写会出错。

索引 i	实际分支	第二个整数
0	case 5	$[5x + \texttt{0x4f19}]_{32}$
1	case 0	$[3x + \texttt{0x3a75}]_{32}$
2	case 1	$[(x \oplus \texttt{0x643}) + (x \gg 1)]_{32}$
3	case 3	$[4x]_{32} \oplus \texttt{0x7b03}$
4	case 6	$[(x \oplus \texttt{0xd854}) + (x \gg 1)]_{32}$
5	case 7	$[4x]_{32} \oplus \texttt{0x7d53}$
6	case 4	$[5x + \texttt{0x26cb4}]_{32}$
7	case 2	$[5x + \texttt{0x4385a}]_{32}$

代码先输出 i，再用 switch 计算第二项。表格中的常量来自个人炸弹的实际指令，保存的是题目规则，不是几个 seed 的现成答案。例如右移与异或相加的分支，需要保留原来的 x；不能先覆盖 x，再拿改变后的值参与另一半运算。

执行第 3 关公开测试，10 个 seed 全部通过。

```
yufan@yufan-VMware-Virtual-Platform:~/ICS/BombLab$ python3 public_test
.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 6：第 3 关公开测试通过 10/10。

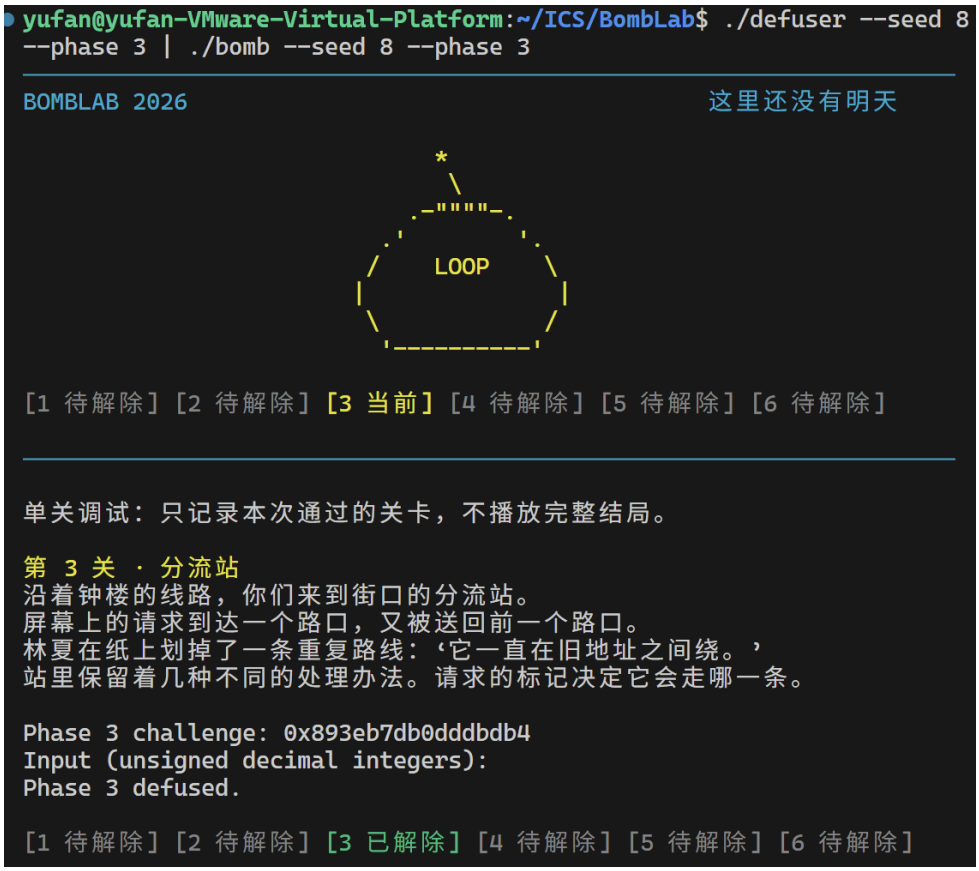


图 7：第 3 关使用非公开 seed 8 运行，输出 `Phase 3 defused.`

第 4 关：递归查找与逐层返回

本关输出一个整数。入口设置目标、密钥和初始区间：

$$x = c \,\&\, 127, \qquad k = [c \gg 7]_{32} \oplus [c \gg 32]_{32}\,, \qquad [l, r] = [0, 127].$$

helper 每次取区间中点，再选择包含目标的那一半。

```
402328: sub %esi,%edi
40232a: shr $1,%edi
40232c: add %esi,%edi
40232e: cmp %eax,%edi
402330: je 402380 <p4_walk+0x60>
402337: lea (%rdi,%rcx,1),%ebx
40233e: cmp %edi,%eax
402340: jb 402360 <p4_walk+0x40>
402347: call 402320 <p4_walk>
40234c: lea (%rax,%rax,4),%eax
402356: xor $0x17,%eax
402365: call 402320 <p4_walk>
402374: xor $0x1f,%eax
402380: lea 0x68(%rdi),%eax
```

设当前区间的中点为

$$m = l + \left\lfloor \frac{r - l}{2} \right\rfloor .$$

将递归函数记为 $F(x,l,r;k)$，三种返回情况分别为

$$F(x,l,r;k)=\begin{cases}[m+\texttt{0x68}]_{32}\oplus k, & x=m,\\ [5F(x,l,m-1;k)]_{32}\oplus[m+k]_{32}\oplus\texttt{0x1f}, & x<m,\\ [5F(x,m+1,r;k)]_{32}\oplus[m+k]_{32}\oplus\texttt{0x17}, & x>m.\end{cases}$$

本关口令为 $F(x,0,127;k)$。其中左、右分支使用的常量分别是 `0x1f` 和 `0x17`，递归过程中 k 始终不变。

因此只找到目标位置还不够，必须按调用栈由内向外合并返回值。实现用 `walk4` 保留相同递归结构，左右分支分别写清参数与常量。目标原本就在合法区间内，且每次区间缩小，因此会到达命中条件；不需要增加与实际程序无关的空区间返回规则。

执行第 4 关公开测试，结果为 10/10。

```
yufan@yufan-VMware-Virtual-Platform:~/ICS/BombLab$ python3 public_test
.py --phase 4
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 8：第 4 关公开测试通过 10/10。



图 9：第 4 关使用非公开 seed 8 运行，输出 `Phase 4 defused.`

第 5 关：字段偏移与链表遍历

本关要求两个整数，分别是累计结果和最后访问节点的 tag。节点区域起点是 `0x405ba0`，相邻节点相距 16 字节：偏移 0 为 value，偏移 4 为 tag，偏移 8 为 next 指针。片段中的 `0x405b60` 是八项头节点索引表，不是节点区本身。

```
40241d: and $0x7,%eax
402420: shr $0x3,%rcx
402424: mov 0x405b60(,%rax,8),%rdx
402433: shr $0x6,%rax
402437: xor %edi,%eax
402439: mov 0x4(%rdx),%esi
40243c: mov %ecx,%edi
40243e: xor $0xbf49,%eax
402443: lea (%rax,%rax,2),%eax
402446: add (%rdx),%eax
40244b: xor %esi,%eax
40244d: and $0x3,%edi
402457: mov 0x8(%rdx),%rdx
402461: mov 0x4(%rdx),%esi
402466: xor %esi,%eax
```

将头节点索引表记为

$$H = (11, 9, 3, 5, 6, 10, 1, 0).$$

令 p_j 表示第 j 次访问的节点索引，V_p、T_p、N_p 分别表示节点 p 的 value、tag 和 next 字段。初始

节点、访问次数和累计值为

$$\begin{aligned} p_0 &= H_{c\&7}, \\ n &= 1 + ((c \gg 3)\ \&\ 3), \\ s_0 &= [c \gg 6]_{32} \oplus [c \gg 32]_{32} \oplus \texttt{0xbf49}. \end{aligned}$$

每访问一个节点，就更新累计值并沿 next 移动：

$$\begin{aligned} s_{j+1} &= \left[3s_j + V_{p_j}\right]_{32} \oplus T_{p_j}, \\ p_{j+1} &= N_{p_j}, \qquad j = 0, \ldots, n-1. \end{aligned}$$

两个输出依次为 s_n 和 $T_{p_{n-1}}$，即累计结果与最后一个已访问节点的 tag。

头节点已经在循环前处理了一次，之后最多再走三次，所以 $n \in \{1, 2, 3, 4\}$。代码用结构体数组保存个人炸弹的数据，用索引表达 next，避免将程序中的固定地址直接当成自己进程里的指针。

例如物理索引 11 的 next 是 9，接着是 3、5；它不是按节点在文件里的物理位置顺序移动。当前头表和步数范围保证访问完成前不会遇到 NULL。完整的十二个节点 value、tag、next 保存在 `defuser.c` 的 nodes 数组中。

```
yufan@yufan-VMware-Virtual-Platform:~/ICS/BombLab$ python3 public_test
.py --phase 5
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 10：第 5 关公开测试通过 10/10。



图 11：第 5 关使用非公开 seed 8 运行，输出 `Phase 5 defused.`

第 6 关：函数指针表与两次间接调用

本关输出一个整数。入口把 challenge 交给 `p6_dispatch`，后者从函数指针表中选择两次操作，第一次结果还要经过异或才能作为第二次的输入。

```
40256c: shr $0x6,%rdi
402573: shr $0x20,%rax
402577: xor %eax,%edi
402579: mov %rbx,%rax
402580: and $0x38,%ebx
402583: and $0x7,%eax
402586: call *0x405e00(,%rax,8)
40258d: xor $0xcb24,%eax
402592: mov %eax,%edi
402594: call *0x405e00(%rbx)
40259f: add $0x953e,%eax
```

函数表位于 `0x405e00`，其映射关系及两次索引为

$$P = (1, 3, 2, 2, 4, 3, 0, 5),$$
$$i_0 = c \,\&\, 7, \qquad i_1 = (c \gg 3) \,\&\, 7.$$

汇编保存的 $c \,\&\, \texttt{0x38}$ 是第二个表项的字节偏移，与 $8i_1$ 等价。将六种操作记为 $f_0, \ldots, f_5$，对应关系如下。

操作编号	计算关系	观察重点
0	$\mathrm{ror}_{32}(x, 7)$	右侧移出的位回到左侧
1	$[2x + \texttt{0xb13c}]_{32} \oplus [7x]_{32}$	两个表达式都使用原来的 x
2	$[(x \oplus (x \gg 5)) + \texttt{0xf09a}]_{32}$	异或后再加常量
3	$[x + \texttt{0x484b}]_{32} \oplus (x \gg 5)$	加法与右移分别计算
4	$\mathrm{rol}_{32}(x, 1) \oplus \texttt{0x7481}$	左侧移出的位回到右侧
5	$[(x \oplus \texttt{0x85d4}) + 4x]_{32}$	先异或，再与原值四倍相加

两次间接调用与最后的加法可写成

$$
\begin{aligned}
x_0 &= [c \gg 6]_{32} \oplus [c \gg 32]_{32}\,, \\
x_1 &= f_{P_{i_0}}(x_0) \oplus \texttt{0xcb24}, \\
v &= \left[f_{P_{i_1}}(x_1) + \texttt{0x953e} \right]_{32}.
\end{aligned}
$$

实现用 `phase6_op` 的 switch 表达六种操作，再由 ops 数组恢复实际表序。这里 ror_{32}、rol_{32} 分别表示 32 位循环右移、循环左移；循环移位用两次方向相反的移位和按位或实现，普通移位不能代替循环移位。

执行第 6 关公开测试，10 个 seed 全部通过。

```
yufan@yufan-VMware-Virtual-Platform:~/ICS/BombLab$ python3 public_test
.py --phase 6
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 12：第 6 关公开测试通过 10/10。

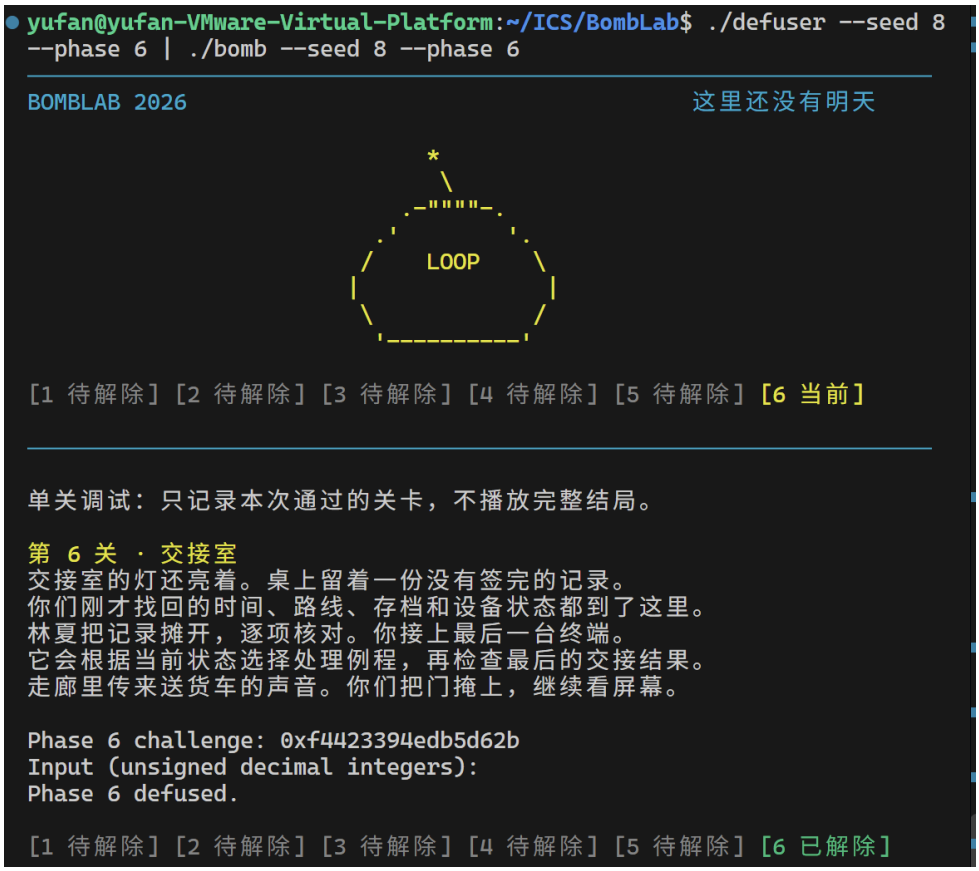


图 13：第 6 关使用非公开 seed 8 运行，输出 `Phase 6 defused.`

Honor：本地对象布局与第七行文本

Honor 接收 challenge 和由前六行计算出的 context。入口不只是读取三个整数，还会先检查整行长度、复制文本、检查对象内的相邻字段，再从后续位置解析整数。

```
40272b: lea -0xd0(%rbp),%r12
402751: sub $0x24,%rax
402755: cmp $0x7f,%rax
402759: jbe 402770 <secret_gate+0x60>
402773: mov %r12,%rdi
402776: call 4026f0 <secret_copy>
40277b: cmpl $0x4e45504f,-0xb0(%rbp)
40278d: lea -0xac(%rbp),%rdi
402794: call 402630 <secret_phase>
40268b: cmp $0x3,%eax
402614: lea 0xfca2(%rdi,%rdi,4),%eax
40261b: xor $0x525b,%eax
```

复制起点为 `rbp-0xd0`，被比较字段在 `rbp-0xb0`，相差 32 字节。按小端字节序，`0x4e45504f` 对应 `OPEN`。随后送入 `secret_phase` 的地址又向后移动 4 字节，所以文本结构为“32 个填充字符 + OPEN + 三个整数”。长度检查允许 36 到 163 字节；当前构造的文本在此范围内。

设 h 为 context，并令 k 对应代码中的 kind。三个整数按以下关系生成：

$$
\begin{aligned}
k &= h \,\&\, 3, \\
a &= [c]_{32} \oplus [c \gg 32]_{32}\,, \\
b &= [h]_{32} \oplus [h \gg 32]_{32}\,, \\
x &= [5(a \oplus b) + \texttt{0xfca2}]_{32} \oplus \texttt{0x525b}, \\
y &= x \oplus [k \cdot \texttt{0xfca2}]_{32} \oplus [h \gg 16]_{32}\,, \\
z &= y \oplus [b + k + \texttt{0x525b} + (a \gg 7)]_{32}\,.
\end{aligned}
$$

整数顺序为 k、y、z。context 必须使用当前六行输出对应的值，不能固定为某次运行的常数；现有驱动已完成这一步并把它作为实参传入。实现用 `snprintf` 构造第七行，并检查返回长度，保证输出缓冲区没有截断。

执行 Honor 公开测试，10 个 seed 全部通过。该测试先验证前六行，再验证对应的第七行，因此也检查了前六行与 context 的配合。

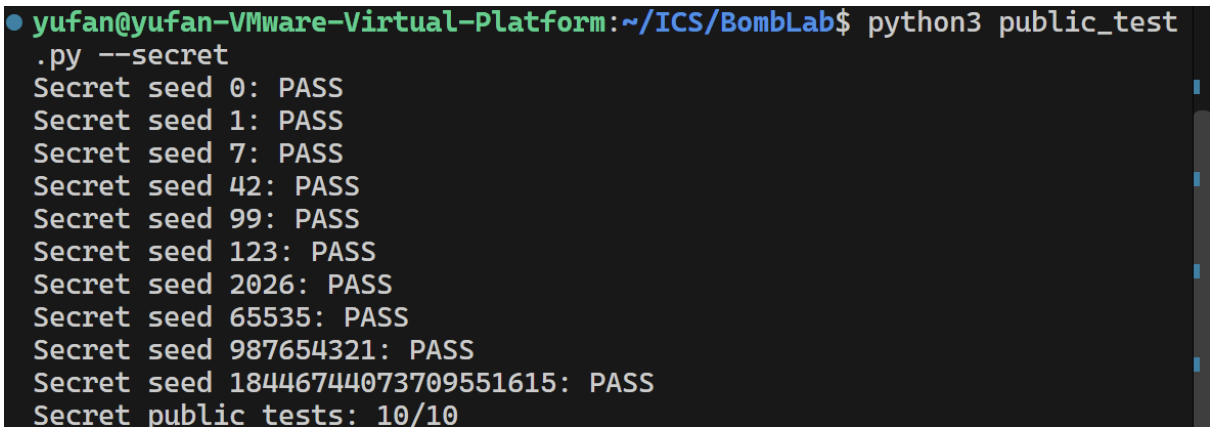


图 14：Honor 公开测试通过 10/10。

再使用公开列表之外的 seed `8` 验证 Honor。先输出前六关答案，再输出对应的第七行文本，将两部分一并交给相同 seed 的炸弹：

```
{
    ./defuser --seed 8
    ./defuser --seed 8 --secret
} | ./bomb --seed 8
```

图 15 记录了这次运行的命令与启动画面；成功结果另经真实 Ubuntu 终端复核，日志末尾为：

```
Congratulations! You've defused the bomb.
Secret phase defused.
```

这说明非公开 seed 下，前六关与 Honor 的第七行能够连续通过。完整复核输出随报告保存在 `logs/honor-seed8.log`。



图 15：非公开 seed 8 的完整运行命令与启动画面。

三、验证过程与结果

代码在真实 Ubuntu 中按课程原有编译选项完成构建。运行单关公开测试和 `public_test.py` 总测试，常规六关通过 60/60，Honor 通过 10/10，结果见图 1 至图 16。

本次截图记录了非公开 seed `8` 在第 1 至第 6 关的单关运行结果（图 3、图 5、图 7、图 9、图 11、图 13）。公开 seed 列表见 `tests/public.json`，`8` 不在其中，这些试跑属于公开列表之外的额外验证。Honor 同样使用非公开 seed 8 完成验证，命令见图 15，通过结果见上一节的终端日志摘录。

关卡	验证记录	实验截图
第 1 关	公开 10/10；seed 42、seed 8 复跑	图 1、图 2、图 3
第 2 关	公开 10/10；非公开 seed 8 复跑	图 4、图 5
第 3 关	公开 10/10；非公开 seed 8 复跑	图 6、图 7
第 4 关	公开 10/10；非公开 seed 8 复跑	图 8、图 9
第 5 关	公开 10/10；非公开 seed 8 复跑	图 10、图 11
第 6 关	公开 10/10；非公开 seed 8 复跑	图 12、图 13
Honor	公开 10/10；非公开 seed 8 通过	图 14、图 15
总测试	常规六关 60/60；Honor 10/10	图 16

四、关键问题与理解

分析的第一步是把输入数与检查条件对应起来，并跟踪 helper 调用前后的参数和返回值。第 3、6 关的表项可能经过重排，因此表索引不等于函数编号；第 5 关中，节点的物理排列也不等于 next 所决定的访问顺序。第 4 关的返回值是在递归返回时逐层合成，而不是只由最终目标决定。

位宽同样需要单独检查。64 位 challenge 的移位必须发生在截成 32 位之前，循环移位与普通移位不同，32 位加乘也不能随意改用有符号整数。修改代码后应重新编译，再使用相同命令运行，以免误把旧可执行文件的结果当作新代码的结果。

五、资料与辅助工具使用

报告使用 LaTeX（XeLaTeX，TeX Live 2025）排版，图片来自本人运行时的终端截图。写作过程中使用 AI 助手协助 LaTeX 排版、汇编核对与最终终端复核。参考资料为课程仓库的 `README.md`、`tutor/` 与 `teaching/` 目录。

附：公开测试总汇总

```
yufan@yufan-VMware-Virtual-Platform:~/ICS/BombLab$ python3 public_test
.py
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

图 16：总测试输出，常规六关通过 60/60，Honor 通过 10/10。

希望我能在看到 `boom!` 和 `PASS` 之后，想起某年某月某天为这个实验伤心和开心了很久之后的自己。

感谢远道而来的实验和助教的用心。疲惫之后，记得跟彼此的自己说一句，早安。